

백승훈: 포트폴리오

웹으로 보시는 것을 더 권장합니다.

bagzaru.github.io

백승훈 | bagzaru



안녕하세요. 게임 개발 전문가를 꿈꾸는 백승훈입니다!

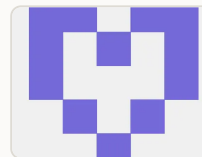
많은 사람들이 게임 속 세상에서 몰입하고
즐겁게 플레이할 수 있는 게임을 만들고 싶습니다!

Contacts

- e-mail: bagzaru3690@gmail.com
- phone: 010-7674-3738

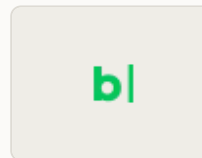
Links

- github:



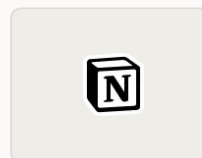
github
github.com

- blog:



blog
blog.naver.com

- devlog:



devlog
bagzaru.notion.site

학력

- 2000년 03월 16일 출생
- 선린인터넷고등학교 웹운영과 졸업
 - 2016.03~2019.02
- 중앙대학교 소프트웨어학부 졸업 예정
 - 2019.03~2026.02
 - 현재 4학년 과정을 마치고, 졸업 예정자 신분입니다.

병역

- 육군 정보통신운용병 병장 전역
 - 2021.03~2022.09

2. 프로젝트 경험

2.1. 게임 프로젝트

2.2. 버전 관리 도구 프로젝트

2.1. 게임 프로젝트

2.1.1. Maneuver

2.1.2. Camping Alone VR

2.1.3. BeThePlayer

2.1.4. 선린 게임프레임워크

Maneuver



링크



Steam: Maneuver
store.steampowered.com

언어 및 도구

게임 엔진 및 언어: Unity, C#
버전 관리: Git, Github

개발 기간

2023년 10월~2024년 1월

맡은 역할

- 근접 공격 및 대쉬 기능 구현
 - 근접 공격의 충돌 판정 구현
 - 낙백 시스템 구현
- Behavior Tree 기반 AI 구현
 - 적 기본 유닛 행동 로직 제작(Behavior Tree)
- 그 외 일부 타격 시스템, 전투 VFX, 맵 제작, 폴리싱 등

소개

Maneuver는 적을 해킹하여 계속해서 몸을 바꿔가며 전투하는 1인칭 3D ‘해킹’ 액션 게임입니다. 크래프톤 정글 게임랩 1기 과정 중 제작하였고, Steam에 무료 게임으로 등록하였습니다.

개발 목적 및 성과

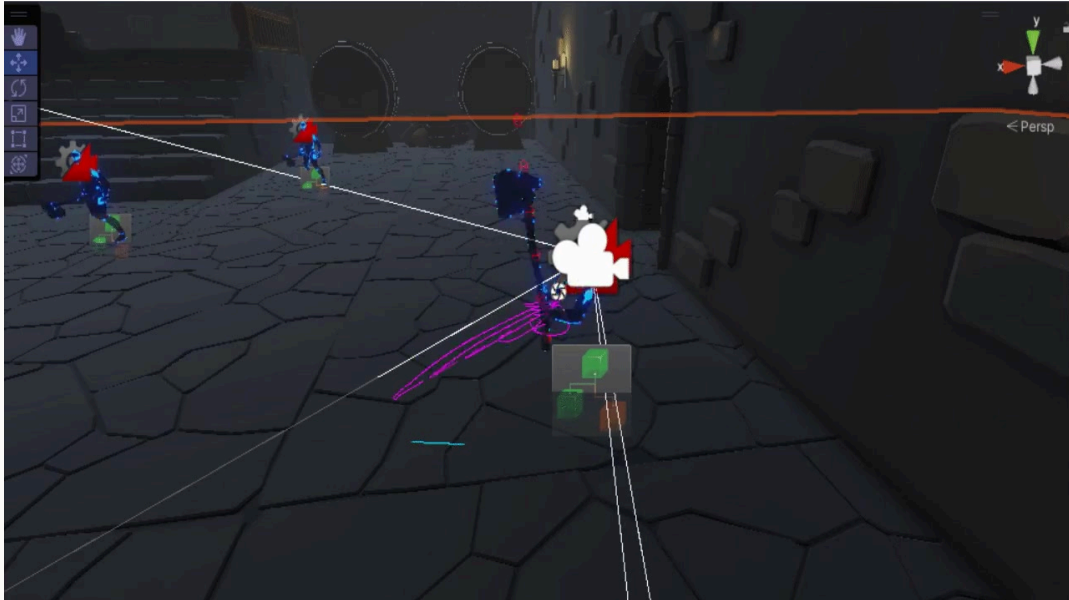
크래프톤 정글 게임랩 1기 과정에 참여하며 제작하였던 게임입니다. 5명의 팀원들과 기획, 개발하였습니다.

이 프로젝트를 통해 협업에 있어서 발전할 수 있었습니다. 팀워크를 위해서는 다른 사람의 말을 존중하되 필요할 때에는 소신 있게 행동할 줄도 알아야 한다는 것을 배웠습니다.

처음으로 스팀에 게임을 출시하며 게임 제작 프로세스를 간접적으로 체험해보는 경험이 되었습니다.

타격 판정 통과 버그

직면했던 문제상황 - 타격 판정 통과 버그



문제 상황

무기에 충돌체를 적용하여 근접 공격을 구현했지만, 저희가 사용하는 애니메이션 애셋의 프레임이 적어 종종 충돌체가 적을 통과하여 타격이 무시되는 일이 발생하였습니다.

해결 과정

처음에는 단순히 플레이어 앞쪽 히트박스를 스캔하여 해결하려고 했습니다.

하지만 생각보다 판정이 정확하지 않았고, 개발 과정에서 공격의 방향성이 필요해졌습니다.

무기에 여러 검사 지점을 배치한 뒤, 각 지점의 이전 프레임 위치와 현재 프레임 위치에 RayCast를 실행하여 피격을 검출하는 방식으로 문제를 해결하였습니다.

몇 가지 방법을 고안했었지만, 가장 짧은 시간 내에 구현 가능하다고 판단하여 구현하였고,

추가로 당시 1인칭 시점에서 실제 무기 크기보다 더 긴 타격 범위가 필요하다는 피드백이 있었는데, 그를 보완하기에도 적절하다고 판단하여 적용하였습니다.

결과

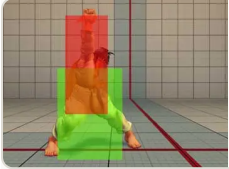
프로젝트 종료 시까지 통과 버그가 다시 발생하지 않았고, 적의 크기, 플레이어/AI 여부에 따라 쉽게 피격 범위를 수정하여 사용할 수 있게 되었습니다.

관련 기록



W11: Warhammer 액션 분석

bagzaru.notion.site



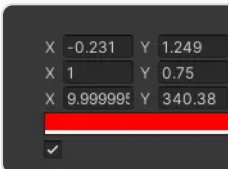
W11: 근접 공격 구현(HitBox)

bagzaru.notion.site



W11: Attack Hit Box 컴포넌트

bagzaru.notion.site

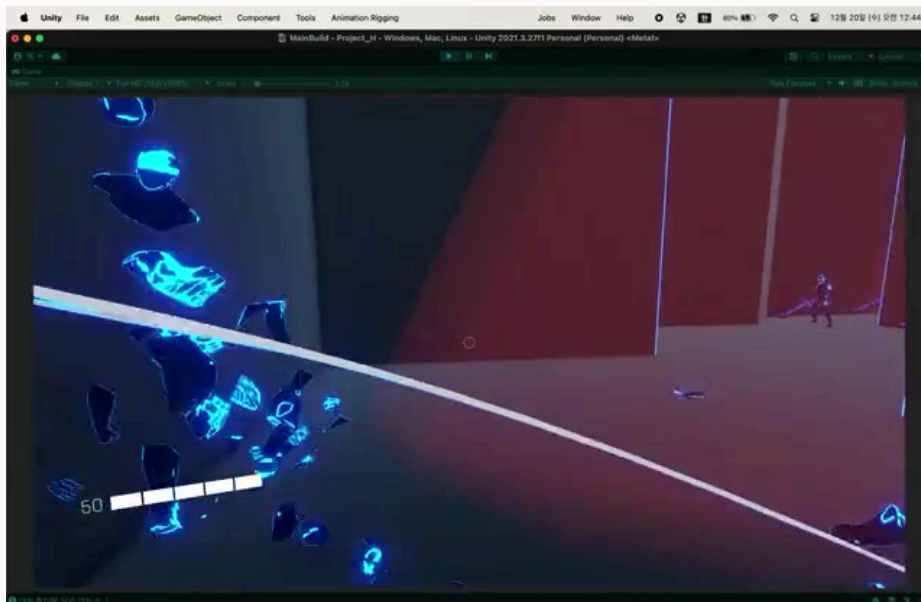


W14: 보간을 통한 충돌 범위 감지

bagzaru.notion.site

AI 움직임 개선

직면했던 문제 상황 - AI 움직임 둔화



문제 상황

적 AI가 좁은 길이나 적이 많을 때 느리게 걸어오는 문제가 있었습니다.

최종 빌드 마감까지 2일 남은 상황에 발견한 문제였습니다. (릴리즈 버전에서만 발생하는 문제였습니다.)

저희 게임은 속도감을 최우선으로 생각했기에, 반드시 해결해야하는 문제였습니다.

구현 과정



여러 테스트 끝에 Obstacle Avoidance가 High Quality일 때 발생하는 문제라는 것을 확인하였습니다.
해당 값이 커져있으면 다른 부분에서 어색한 움직임이 나타났고, None이 되면 적들끼리 겹치는 문제가 발생했습니다.

Docs나 웹에서 Obstacle Avoidance에 대한 정보를 충분히 찾지 못하였기 때문에, 우선 NavMesh의 기능을 통해 Path만 알아내고, 이를 Rigidbody의 Move로 이동하는 방식을 해결을 시도하였습니다.

그러나 적이 뭉쳐있을 때 여전히 자연스럽지 않고, 기존 OffMeshLink를 처음부터 다시 구현하는 문제, AI끼리 충돌 시 부자연스럽게 밀리는 문제가 추가적으로 발생했습니다.

해당 안은 빠르게 폐기하고, 다른 안인 주변을 매 사이클 검사해서 적이 있으면 Obstacle Avoidance를 High로, 적이 없으면 None으로 두는 방식을 적용하였습니다.

실제로 적용해보니 공격 중에 적이 움직이는 문제가 발생하였지만, 이전 실험에서 None은 High를 무시하지만 High는 None을 보고 피한다는 점을 이용해 공격 중일때에만 None으로 강제하는 방식으로 구현하여 문제를 해결하였습니다.

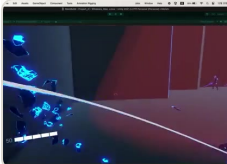
사실 여전히 길이 매우 좁으면서 적이 아주 많이 뭉쳐있으면 느려지는 문제가 발생할 가능성이 존재하였습니다.(적이 너무 많아 강제로 좁은 길에서 High로 돌아갔기 때문이었습니다.) 하지만 저희 게임에서 해당 지형은 아주 잠깐 등장하였기에, 해당 지형에서 적의 밀도를 줄이고, 여러번의 테스트를 통해 플레이 시 자연스럽게 수정하여 문제가 거의 느껴지지 않게 수정했습니다.

후기

조금 더 시간이 있었더라면 더 많은 문서를 찾고, 더 많은 실험을 진행하여 더 나은 해결 방법을 찾을 수 있었을 것 같아 아쉽게 느껴집니다.

그럼에도 제한된 시간 동안 문제가 거의 느껴지지 않게 수정했던 것 같아 다행이었다고 생각합니다.

당시 작성한 문서



23.12.29 NavMesh중 코너에 가면 속도가 느려지고 이상하게 움직임

bagzaru.notion.site

그 외 맡은 역할들

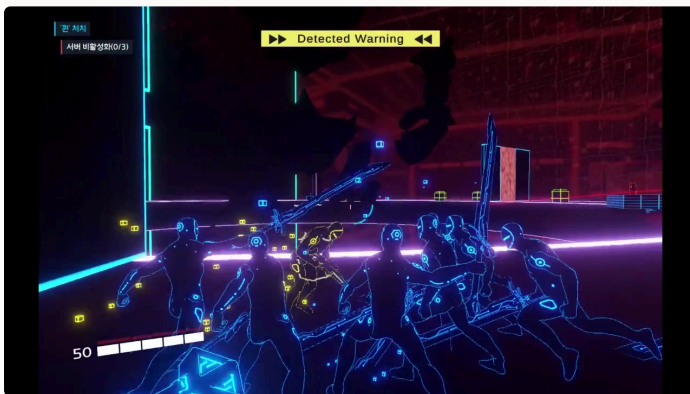
대쉬 기능 구현

대쉬 기능을 구현하였습니다.



무기, 총알 VFX 구현

Unity의 Visual Effect Graph를 이용하여, 근접 공격, 샷건의 파티클과 트레일 이펙트를 구현하였습니다.



적 AI 구현

Behavior Tree를 활용하여, 전반적인 적 AI 제작에 참여하였습니다.

애니메이션 기반 공격 상태 제어

애니메이션 이벤트를 이용해, 근접 공격을 '공격 준비', '공격 중', '공격 종료' 상태로 나누고 이를 기반으로 데미지 처리, 다음 입력 허용 등 로직을 구현하였습니다.

넉백 기능 구현

플레이어의 넉백은 물리 기반 AddForce(Impulse)를 이용해 구현하였습니다.

AI의 넉백은 NavMesh 이동과 물리 이동을 모두 고려하여, 즉시 NavMeshAgent를 종료, Behavior Tree 및 Animator에 넉백 상태를 전달하여 넉백 순간에는 물리를 따라 이동하게 수정하였습니다.

땅에 떨어지기 전에 넉백이 끝나 갑자기 땅에 붙는 예외를 처리하기 위해 넉백 최소 시간이 지나면서 땅에 바닥이 닿았을 때에만 넉백이 해제되도록 로직을 작성하였습니다.

Camping Alone VR



언어 및 도구

게임 엔진 및 언어: Unreal, C++
버전 관리: Perforce
특이사항: VR 환경

개발 기간

2025년 11월~2025년 12월

맡은 역할

- 이동 구현
 - 이동, 달리기 구현
- 단검을 이용한 행동 구현
 - 도축 구현
- 고기 구현
 - 고기 마블링(Material) 구현

소개

VR기기를 이용해 활을 쏘아 동물을 사냥하고, 칼로 동물을 도축하고, 고기를 획득해 오래 살아남는 게임입니다.

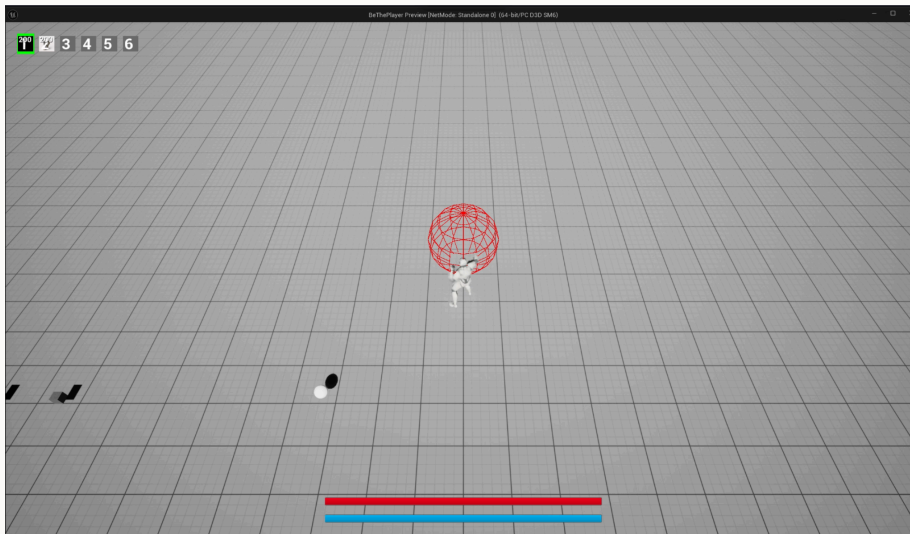
Unreal Engine 기반으로 제작되었고, VR 기기로 플레이하는 게임입니다.

개발 목적 및 성과

2025-2학기 가상증강 혼합현실 과목 최종 프로젝트로 작업한 게임입니다.

Unreal Engine과 가상현실에 대한 공부를 진행하며 작업하였던 게임입니다.

BeThePlayer



언어 및 도구

게임 엔진 및 언어: Unreal, C++
 버전 관리: Git, Github
 특이사항: Unreal GAS, OpenAI Codex, Claude Code

개발 기간

2026년 6월~

소개

현재 진행중인 개인 프로젝트입니다.

AI를 활용하여 개인 프로젝트로 패링 기반의 3D 쿼터뷰 소울라이크 스타일 게임을 제작 중에 있습니다.

무기가 각각의 체력, 스테미나를 가지는 태그매치 형태로 진행되는 게임입니다.

주요 구현 사항

- GAS 기반 전투 시스템
- 패리
- 인벤토리/아이템 시스템
- 무기별 AbilitySystemComponent 적용, 무기 스위칭 기능 구현
 - 공격, 피격, 애니메이션, VFX 등 실제 행위와 관련된 부분은 Player의 ASC에서 처리
 - GameplayEffect, AttributeSet 등 능력치 기반 로직은 Weapon의 ASC에서 처리
 - WeaponManagerComponent가 두 ASC간 연결을 담당
- 어빌리티 별 실행 정책
 - Queued: 현재 행동 유지, 바로 실행 불가 시 대기, 큐 소유권 관리(소유권 획득 시 다른 Queued는 대기 불가)
 - Interrupt: 현재 어빌리티가 중단 가능할 경우, 현재 행동 큐를 비우고 즉시 실행 시도
 - Parallel: 현재 행동과 관계없이 동시에 실행 시도
 - SystemForced: 죽음 등 다른 어빌리티를 반드시 취소할 수 있는 어빌리티

AI 개발 루프

BeThePlayer: AI 활용 개발 루프

현재는 다음과 같은 방식으로 게임을 개발하고 있습니다.



1. 추가할 기능에 대한 **설계 문서 작성**
 - 작성 후 결함 여부를 AI와 검토하여 문서 완성
2. 설계 문서 기반으로 **구현 명세 문서 제작**
 - 설계 문서의 내용을 주요 기능별로 분류합니다.
 - 구현 순서를 정의합니다.
3. **AI 구현 시작**
 1. 2에서 작성한 명세 문서 기반 작업 진행
 2. git worktree를 통한 작업 공간 분리
 3. 코드 리뷰 루프 수행 후 문제가 없으면 작업 완료
4. 수정된 코드를 **직접 검토**
 1. 이 과정에서 제가 예상한 방향성과 다른 부분이 있으면 구현한 이유 확인 후 수정을 진행합니다.
5. 문제가 없다고 판단하면 코드를 작업 브랜치에 병합하고, 워크트리를 정리합니다.

언리얼과 AI

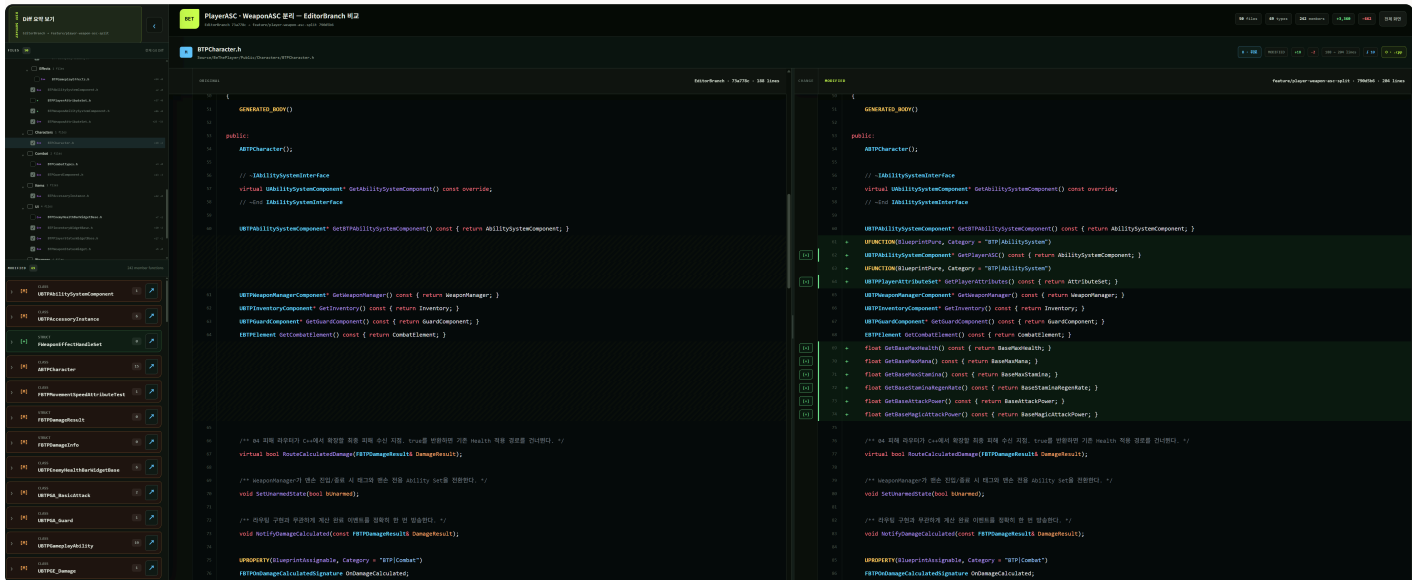
AI에게는 최대한 C++ 코드 위주의 작업을 맡깁니다.

Unreal MCP가 있긴 하지만 아직 불편함이 있고, 결국 에디터 작업은 코드 기반의 작업 위에서 값을 커스터마이징하며 실제 게임을 만드는 부분이라고 생각하여 에디터 작업은 제가 진행합니다.

대신 블루프린트 그래프나 에디터 기반 작업을 최대한 배제하고, 대부분의 기능을 C++에서 구현하는 방식을 사용하고 있습니다.

Diff Tool Build

BeThePlayer: Diff Tool Build Skill



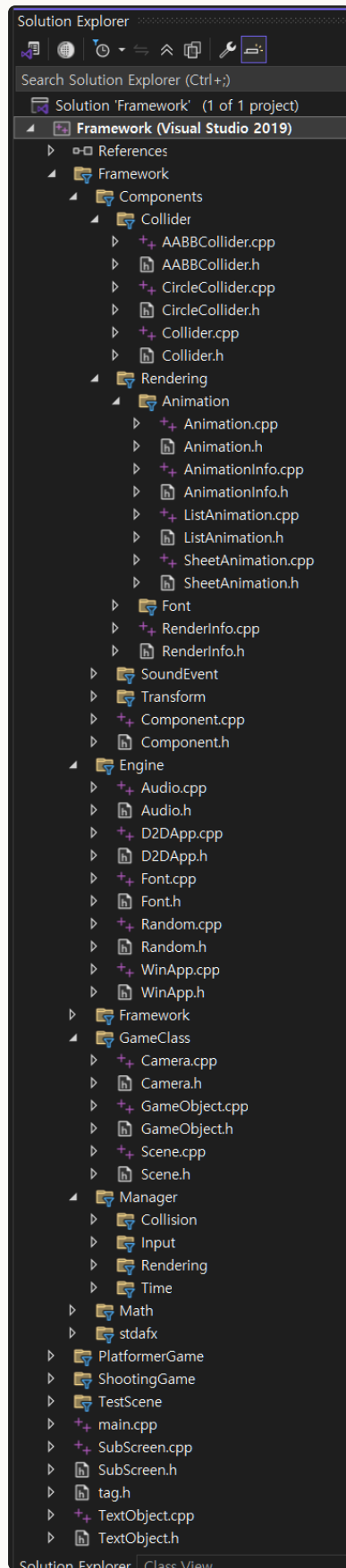
현재 Rider IDE를 이용하여 개발 중에 있습니다. 하지만 Rider의 Git Diff에서는 새로 생성된 파일에 대해 구문 강조 기능을 제공해주지 않고, 헤더/정의간 스위칭이나 선언/정의로 이동 등 기능을 제공하지 않아 변화된 코드를 읽는데 약간의 불편함이 있습니다.

이를 해결하기 위해 AI를 활용해 자체 Diff Tool을 빌드하여 Diff를 확인하고 있습니다. 새로 생성되거나 수정된 함수 간략 설명, 헤더/정의 파일간 스위칭, 주요 흐름 정리 등을 한눈에 볼 수 있는 사이트를 빌드하는 기능을 넣어 코드 변화를 빠르게 확인합니다.

저는 AI가 코드를 짜주는 시대에도 여전히 프로그래머는 코드의 흐름과 주요 지점들을 이해하고 있어야 한다고 생각합니다.

짧은 호흡에서는 코드를 이해하는 시간적 병목이 생길 수 있지만, 긴 호흡의 개발에서는 코드에 대한 이해가 존재해야 문제가 생겼을 때 더 빠르고 잘 대처할 수 있다고 생각합니다.

선린 게임프레임워크



```
void Framework::StartGameLoop()
{
    MSG msg;
    ZeroMemory(&msg, sizeof(MSG));

    while (msg.message != WM_QUIT) {
        if (PeekMessage(&msg, 0, 0, 0, PM_REMOVE))
        {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }

        //Check Input
        TimeManager::UpdateTime();
        InputManager::UpdateKeyState();
        //Update World
        Scene::currentScene->UpdateGameObjects();
        Scene::currentScene->UpdatePhysics();
        Scene::currentScene->DeleteDestroyedObjects();
        Scene::currentScene->Render();
        //Change Scene
        Scene::SwapScene(d2dApp);
    }
}
```

```
void Framework::Run(Scene* startScene, const wchar_t* title, int width, int height, bool isFullScreen)
{
    Random::GetInstance();
    if (SUCCEEDED(coInitialize(NULL)))
    {
        if (winApp->Initialize(title, width, height, isFullScreen))
        {
            if (d2dApp->Initialize())
            {
                bool audioInitialized = Audio::GetInstance()->Initialize();
                InputManager::winApp = winApp;
                Scene::ChangeScene(startScene);
                Scene::SwapScene(d2dApp);
                StartGameLoop();
                SAFE_DELETE(Scene::currentScene);
                SAFE_DELETE(Scene::nextScene);
                if (audioInitialized)
                {
                    Audio::GetInstance()->Uninitialize();
                    d2dApp->Uninitialize();
                }
            }
        }
        CoUninitialize();
    }
}
```

```
void Scene::Render()
{
    renderingManager->BeginRender();

    Vector2 screenSize;
    screenSize.x = WinApp::GetScreenWidthF();
    screenSize.y = WinApp::GetScreenHeightF();
    for (auto& i : renderableList)
    {
        i->renderer->Render(Scene::d2dApp, i->transform, i->renderer->ComputeWorldPosition(
            screenSize, i->transform, camera->transform->position));
    }

    for (auto& i : uilist)
    {
        i->renderer->Render(Scene::d2dApp, i->transform, i->renderer->ComputeUIPosition(i->transform));
    }

    //renderingManager->Render(i->renderer, i->transform, camera->transform->position);
    //renderingManager->Render(i->renderer, i->transform);
    renderingManager->EndRender();
}
```

링크



선린 게임 프레임워크(구글 드라이브)

drive.google.com

언어 및 도구

언어: C++

특이사항: Direct2D

개발 기간

2019년 5월~2019년 11월,

2020년 5월~2020년 11월

소개

선린인터넷고등학교 게임프로그래밍 과목 수업조교로 활동하며, 수업 자료로 사용할 C++, Direct2D 기반 게임 프레임워크를 제작하였습니다.

Direct2D 렌더링, AABB Collider 충돌, 프레임 애니메이션, Input, Scene 등 기본적인 게임 구조를 설명하기 위한 프레임워크를 제작하였습니다.

목표

학생들이 게임을 플레이하거나 Unity 등 게임 엔진 공부를 하기 전에, 게임이 돌아가는 원리와 전체적인 구조를 이해할 수 있게 하는 것을 목표로 작성한 프레임워크입니다.

처음에는 텅 빈 while문부터 시작하여 하나씩 기능을 추가하면서, 오브젝트 관리 및 업데이트, 충돌(AABB 충돌), 키 입력, 2D 렌더링 등을 아우르는 게임 루프를 구현하고, 이를 기반으로 간단한 슈팅 게임을 만드는 과정으로 교육을 진행하였습니다.

배운 점

많은 학생들이 소스코드를 보고 쉽게 이해할 수 있도록 구현하는 것이 목적이었기 때문에, 디커플링과 캡슐화 등 객체 지향적 방향성에 많은 고민을 하고 신경을 썼던 프로젝트였습니다.

학생들과 수업중 제작한 프레임워크를 기반으로 각자의 게임을 개발하는 시간이 있었고, 이 과정에서 내가 보는 코드와 다른 사람이 보는 코드의 깔끔함이 다르게 느껴질 수 있겠다는 사실을 느꼈습니다.

돌이켜 보면 부족함이 많은 프레임워크지만, 이 작업을 기점으로 다른 사람들이 쉽게 이해할 수 있는 코드를 짜기 위해 노력하기 시작했습니다.

2.2. 버전 관리 도구 프로젝트

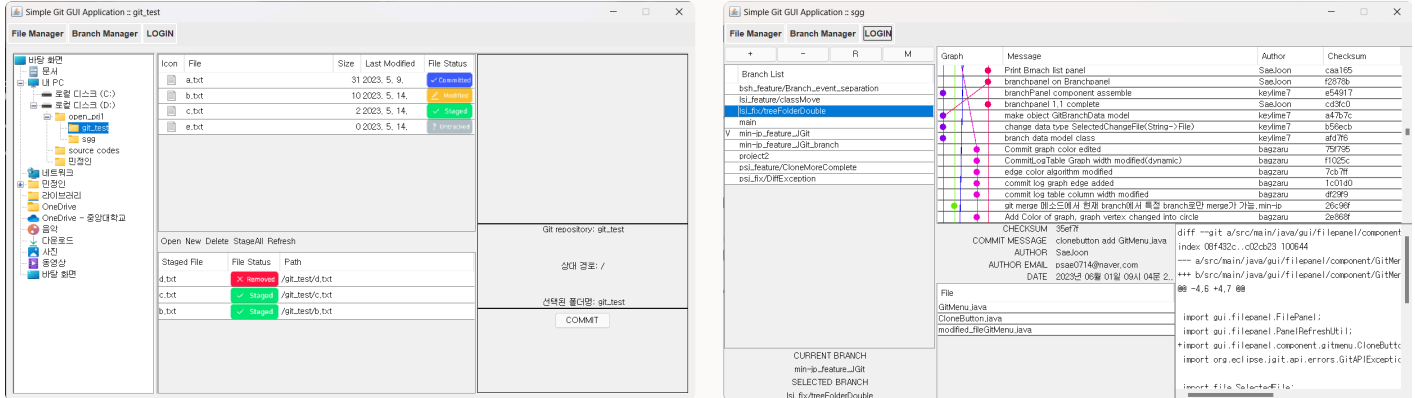
Git, Perforce를 활용한 협업이 가능합니다.

2.2.1. Simple-Git-GUI

2.2.2. Perforce 가이드 문서

Simple-Git-GUI

2.2.1. Simple-Git-GUI



링크

bagzaru/simple_git_gui

github.com

언어 및 도구

언어 및 라이브러리: Java, JGit
버전 관리: Git, Github

개발 기간

2023년 5월~2023년 6월

소개

2023-1학기 '오픈소스 소프트웨어 프로젝트' 과목 과제로 4인 팀으로 GUI로 Git을 관리할 수 있게 하는 프로그램을 구현하였습니다.

JGit 라이브러리를 통해 Git 명령어를 Java에서 실행가능하게 구현, 커밋 그래프 구현, 그외 기타 이벤트 구현을 담당하였습니다.

Git을 활용한 협업을 공부하며 GUI 프로그램을 제작하였습니다.

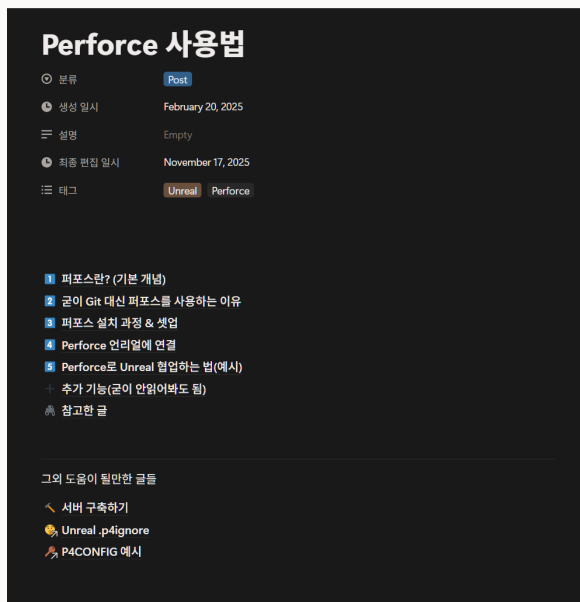
관련 문서



구현 보고서 중 발췌
bagzaru.notion.site

Perforce 가이드 문서

2.2.2. Perforce 가이드 문서 제작



링크



Perforce 사용법

bagzaru.notion.site

2025-1학기 Dwarf Underground, 2025-2학기 CampingAloneVR 프로젝트 진행 시 팀원들을 빠르게 적응시키기 위한 Perforce 가이드 문서를 작성하였습니다.

가이드를 작성하면서 Perforce에 익숙해질 수 있었습니다.

걸어온 길

그 외에도, 게임 개발자를 꿈꾸며 여러 활동을 진행해왔습니다.

2016

선린인터넷고등학교 웹운영과 입학

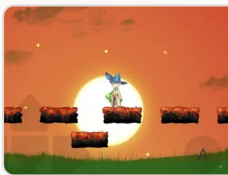
게임개발동아리 "RG" 활동



Revival Gunin

동아리 엔진 기반 C++ 2D 탄막 슈팅 게임.
- 탄막 구현, 보호막 구현 담당.

2017



Lost Melody

Unity 5.6 기반, 모바일 플랫폼 어드벤처 게임.
- 전체 개발 담당.



Lunaway

C++, OpenGL 기반 3D 러너 게임.
- 전체 개발 담당.

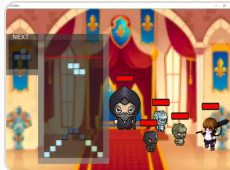


Ten Daze

C++, 동아리 엔진 기반 클릭어 게임.
- 아이템 구현 담당.

2019

중앙대학교 소프트웨어학부 입학



Tetris World

기초 컴퓨터 프로그래밍 과목 과제, C, Direct 2D 기반 테트리스 게임.
- C언어 기반 Direct2D 프레임워크 구현 담당.

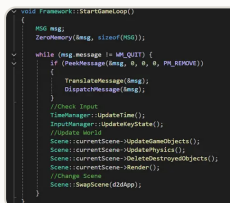
2020



Colorful Bullet Hell

컴퓨터 게임설계 과목 과제, Godot 기반 탄막슈팅 게임.
- 스테이지 2 구현, 탄막 시스템 구현, 데미지 로직 구현.

선린인터넷고등학교 협력강사



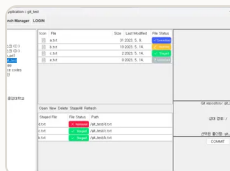
선린 게임프레임워크 (2019~2020)

C++ 기반 2D 게임프레임워크

2021

병역 이행 (2021~2022)

2023

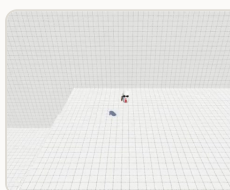


Simple-Git-GUI

오픈소스 소프트웨어 프로젝트 과목 과제.

- Java 기반 Git GUI 툴 제작 담당.

크래프톤 정글 게임랩 1기



Way to Work

3D 플랫폼머 게임.

- 점프, 카메라, 투사체 담당



좁는 닌자

탐류 액션 게임.

- 수리검 구현 담당

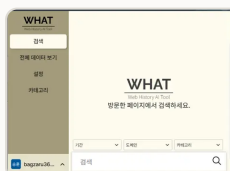


Maneuver

3D 1인칭 액션 게임.

- 근접 전투 시스템, AI 등 구현 담당.

2024



WHAT

AI 인덱싱 기반 방문 기록 크롬 익스텐션.

- 크롬 익스텐션 프론트엔드 구현 담당.

2025



Dwarf Underground

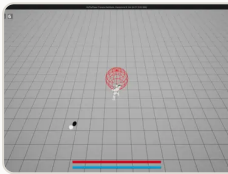
쿼터뷰 실시간 전략 게임.
- UI 구현, 아이템 관련 담당.



Camping Alone VR

VR 1인칭 생존 힐링 게임.
- 이동, 카메라 회전, 단검, 아이템 사용 담당.

2026



2026: BeThePlayer(현재 개발중)

쿼터뷰 액션 소울라이크.
- 전체 개발 담당.

좋아하는 게임

전투를 기반으로 플레이어가 성장하며 모험하는 게임들을 좋아합니다.

가장 재미있게 플레이한 게임



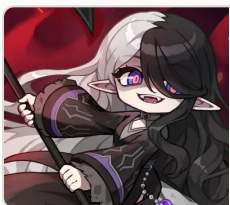
튜닉

store.steampowered.com



엘든 링

store.steampowered.com



메이플스토리

maplestory.nexon.com



33원정대

store.steampowered.com

읽어주셔서 감사합니다.

어떤 일이든 최선을 다하겠습니다.